

# Programmeringsparadigmer 2025

## Kursusgang 10: Funktorer

### Opgaver til løsning og diskussion

Hans Hyttel

11. november 2025

#### Opgaver, som vi helt sikkert vil tale om

1. (*20 minutter*) Typen for ubegrænsede træer `UTree` er givet ved

```
data UTree a = Node a [UTree a]
```

Definér en instans af Functor for `UTree`.

2. (*15 minutter*)

Lad `r` være en given type. Funktionstypekonstruktøren `Arrow-r` er defineret, så `Arrow-r a` bliver `(r -> a)`. Definér en instans af Functor for denne typekonstruktør. *Hint*: Kig på typen af `fmap`.

For at teste din løsning, tilføj følgende i starten af filen, der indeholder din kode:

```
import qualified Prelude
import Prelude hiding (Functor, fmap)
```

3. (*15 minutter*) For den applikative funktor for lister har vi en definition af "sjov stjerne"-sammensætningen `<*>` på side 160. Giv en alternativ *rekursiv* definition af denne, der bruger `fmap`.
4. (*15 minutter*) Brug det faktum, at listetypen kan betragtes som en applikativ funktor til at definere en funktion `prodthree`, der tager tre lister af tal og beregner listen af alle produkter af tripler af tal i listen. Som eksempel, `prodthree [1,2,3] [4,5,6] [7,8,9]` skal give os listen

```
[28,32,36,35,40,45,42,48,54,56,64,72,70,80,90,84,96,108,84,96,108,
105,120,135,126,144,162]
```

*Hint*: Et eller andet sted skinner en sjov stjerne stadig.

#### Flere opgaver til løsning i dit eget tempo

- a) Her er en typedeklaration for simple udtryk.

```
data Exp a = Var a | Val Integer | Add (Exp a) (Exp a) | Mult
  (Exp a) (Exp a) deriving Show
```

Vis, hvordan man gør denne type til en instans af Functor. Hvornår ville det være nyttigt at betragte `Exp a` som en funktor? Tænk på et godt eksempel!

- b) Vis, hvordan man gør typen `Exp` fra det foregående problem til en instans af `Applicative`.
- c) For at løse dette problem skal du allerede have en definition af `Exp` som en applikativ funktor fra problem b. Antag definitionerne

```
type Name = String
type Env = [(Name, Int)]
fetch :: Name -> Env -> Int
fetch x env = case lookup x env of
    Nothing -> error "ugyldigt navn"
    Just v -> v
```

Brug nu alle disse definitioner til at definere en funktion

```
eval :: Exp Name -> Env -> Int
```

som, når den får et udtryk  $e$  og et miljø  $env$ , returnerer værdien af udtrykket, under antagelse af, at alle variable i  $e$  er givet værdier i  $env$ .